appendix A Kubernetes end-to-end example

In this appendix, we will work through an end-to-end observability sample application in Kubernetes. Throughout the book, we've been using Docker Compose as our tool of choice. It has a number of advantages as an educational device; however, in reality, you will more likely use Kubernetes, Amazon ECS, or HashiCorp Nomad as an orchestrator. So this appendix will offer you a more native and consistent approach that covers all the tracing, metrics, and logging topics we discussed and allow you to play around with the setup.

> **TIP** You *could* use `kompose` (https://kompose.io/user-guide/) to turn any of the Docker Compose example files we've covered into Kubernetes resources (e.g., deployments and services) to get a feeling for how to work with the OSS tooling discussed in the book.

Luckily for us, there is already something available that fits the bill: the OpenTelemetry community demo (https://opentelemetry.io/docs/demo/). The demo covers several services written in different programming languages and features the tooling we've discussed. We will first discuss the demo and how it works, then cover the prerequisites to run it, and then walk through the demo together.

A.1 Overview

The OpenTelemetry demo is a microservices-based shopping app that uses 12 different programming languages, including Java, JavaScript, Python, and .NET, to implement services (https://opentelemetry.io/docs/demo/services/). The demo has all its services instrumented, emitting spans, metrics, and log lines. We won't be covering the details of the demo services and their interactions here, but if you're